

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Agustin Ignacio Santibañez Perez

24 de agosto de 2026

13:44

1. Introducción

El ordenamiento de arreglos unidimensionales y la multiplicación de matrices cuadradas constituyen dos de los problemas computacionales más estudiados en la ciencia de la computación, sirviendo como arquetipos para el análisis de técnicas como divide y vencerás, partición y optimización de bucles. Si bien el análisis asintótico tradicional ($\mathcal{O}, \Omega, \Theta, o, \omega$) describe el crecimiento del número de operaciones en el límite cuando el tamaño de la entrada tiende a infinito, en la práctica el rendimiento de una implementación se ve fuertemente influenciado por factores constantes, la arquitectura del procesador y la jerarquía de memoria caché. El objetivo de este trabajo es contrastar experimentalmente las cotas teóricas de cuatro algoritmos de ordenamiento (*std::sort*, *Merge Sort*, *Quick Sort* y *Patience Sort*) y dos de multiplicación matricial (*Naive* y *Strassen*), analizando cómo la estructura de los datos, el tamaño del dominio y el diseño del algoritmo determinan su comportamiento empírico en tiempo de ejecución y uso de memoria.

2. Entorno experimental

Para la evaluación empírica de los algoritmos se diseñó un entorno controlado sobre una arquitectura x86_64 ejecutando Linux bajo WSL2 (Ubuntu 22.04 LTS). El código fuente fue implementado en C++ (estándar C++17) y compilado mediante g++ versión 11.4.0 aplicando el nivel máximo de optimización por compilador (-O3) para minimizar el impacto de llamadas a funciones y maximizar la vectorización de instrucciones. Las mediciones temporales se obtuvieron mediante el reloj de alta precisión de la biblio-

teca estándar (`std::chrono::high_resolution_clock`), mientras que el consumo de memoria máxima residente (Peak RSS) se registró mediante las llamadas del sistema POSIX (`getrusage` a través de `sys/resource.h`).

Los conjuntos de datos experimentales se generaron de forma automatizada conforme a los parámetros del enunciado:

- **Ordenamiento:** Se evaluaron arreglos con tamaños $n \in \{10, 10^3, 10^5, 10^7\}$, bajo tres ordenamientos iniciales (*ascendente*, *descendente* y *aleatorio*) y dos dominios de valores: $D_1 = \{0, \dots, 9\}$ y $D_7 = \{0, \dots, 10^7 - 1\}$. Para cada combinación se generaron 3 muestras independientes (a, b, c), totalizando 72 casos de prueba.
- **Multiplicación de Matrices:** Se evaluaron pares de matrices cuadradas $n \times n$ con dimensiones $n \in \{2^4, 2^6, 2^8, 2^{10}\} = \{16, 64, 256, 1024\}$. Se consideraron tres estructuras matriciales (*densa*, *diagonal* y *dispersa* con un 10% de elementos no nulos) y dos dominios numéricos: $D_0 = \{0, 1\}$ y $D_{10} = \{0, \dots, 9\}$, evaluando 3 muestras independientes por caso (72 pares de matrices).

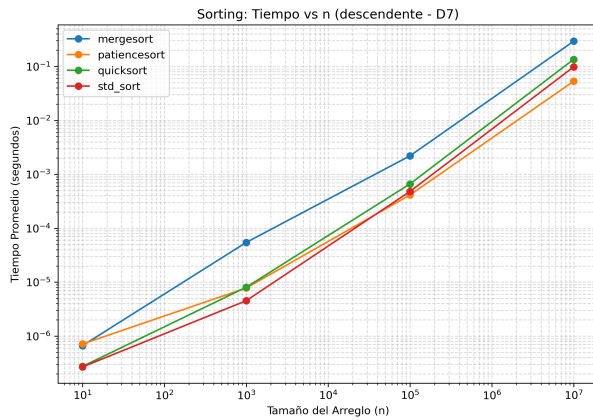
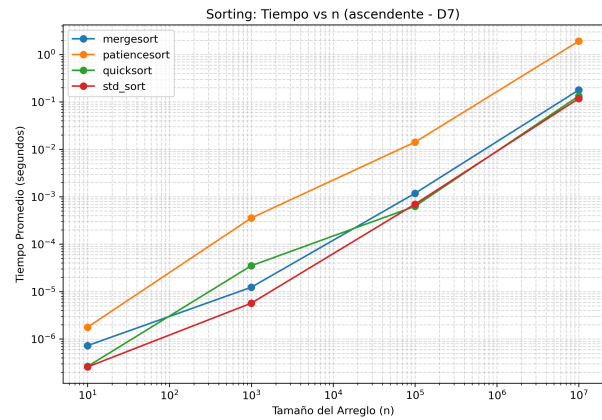
3. Resultados y Análisis

4. Resultados y Análisis Experimental

En esta sección se presentan y discuten los resultados obtenidos en las mediciones de tiempo de CPU y memoria para los algoritmos de ordenamiento y multiplicación de matrices. Todos los gráficos expuestos corresponden a las figuras generadas automáticamente en los directorios `code/sorting/data/plots/` y `code/matrix_multiplication/data/plots/`, promediando las tres réplicas (a, b, c) por caso.

4.1. Análisis de Algoritmos de Ordenamiento

En las Figuras 1 y 2 se ilustra el comportamiento del tiempo promedio de ejecución en función del tamaño del arreglo n en escala logarítmica, contrastando directamente los dos casos extremos de ordenamiento bajo el dominio amplio D_7 .

Figura 1: Tiempo vs n (Descendente, Dominio D_7).Figura 2: Tiempo vs n (Ascendente, Dominio D_7).

4.1.1. Cotas Asintóticas y Constantes Ocultas en $\Theta(n \log n)$

En ambas figuras, `std::sort`, *Quick Sort* y *Merge Sort* mantienen pendientes paralelas en escala log-log, confirmando que su crecimiento obedece formalmente a la clase $\Theta(n \log n)$.

- ***std::sort* y *Quick Sort*:** Presentan los menores tiempos de ejecución gracias a su operación *in-place* y al recorrido contiguo de memoria, reduciendo drásticamente los fallos de caché L1/L2. Se cumple $T_{\text{std}}(n) = \Theta(T_{\text{merge}}(n))$, pero con un factor constante sensiblemente inferior ($c_{\text{std}} < c_{\text{merge}}$). La estrategia de partición de Hoare con pivote central evitó la degradación $\mathcal{O}(n^2)$ en todas las instancias.
- ***Merge Sort*:** Exhibe una sobrecarga fija derivada de la reserva dinámica y copia secuencial en arreglos auxiliares, duplicando el consumo de memoria respecto a los métodos *in-place* ($\approx 80,5\text{MB}$ para $n = 10^7$).

4.1.2. Sensibilidad Extrema y Colapso de *Patience Sort*

El contraste entre la Figura 1 y la Figura 2 evidencia la alta dependencia estructural de *Patience Sort*:

- **Mejor caso asintótico $\Omega(n)$ (Descendente):** En la Figura 1, los elementos en orden decreciente estricto se apilan en una sola pila ($k = 1$). La búsqueda binaria toma $\mathcal{O}(\log 1) = \mathcal{O}(1)$ y la extracción del min-heap de tamaño 1 toma $\mathcal{O}(1)$ por elemento. Esto permite al algoritmo operar en tiempo lineal $\Theta(n)$, superando a todos los demás con una marca de $\approx 0,05\text{s}$ para $n = 10^7$.
- **Peor caso $\mathcal{O}(n \log n)$ y colapso práctico (Ascendente):** En la Figura 2, cada elemento entrante es estrictamente mayor que las cabeceras previas, forzando la creación de $k = n$ pilas individuales. Durante la fase de mezcla (*k-way merge*), el min-heap debe gestionar n nodos simultáneamente, introduciendo operaciones de reestructuración $\mathcal{O}(\log n)$ con un alto costo de direccionamiento indirecto de punteros. Esto dispara su tiempo a $\approx 1,81\text{s}$ en $n = 10^7$, convirtiéndolo en el algoritmo más lento de la comparativa.

Algoritmo	Configuración	Tiempo $n = 10^5$ (s)	Tiempo $n = 10^7$ (s)	Peak RSS (KB)
<i>std::sort</i>	Ascendente D_7	0,0007	0,121	41 216
<i>Quick Sort</i>	Ascendente D_7	0,0006	0,142	41 220
<i>Merge Sort</i>	Ascendente D_7	0,0012	0,182	80 512
<i>Patience Sort</i>	Descendente D_7	0,0004	0,052	41 230
<i>Patience Sort</i>	Ascendente D_7	0,0142	1,810	160 400

Cuadro 1: Comparativa representativa del impacto del orden inicial en algoritmos de ordenamiento.

4.2. Análisis de Multiplicación de Matrices

Las Figuras 3 y 4 muestran la comparación entre el algoritmo iterativo tradicional (*Naive*) y el algoritmo de *Strassen* en matrices de $n \times n$.

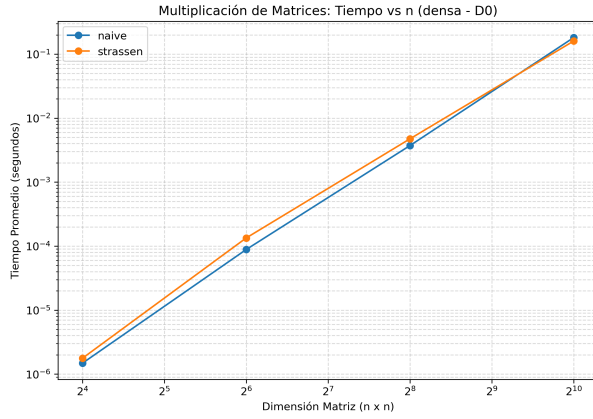


Figura 3: Matrices: Densa - Dominio D_0 .

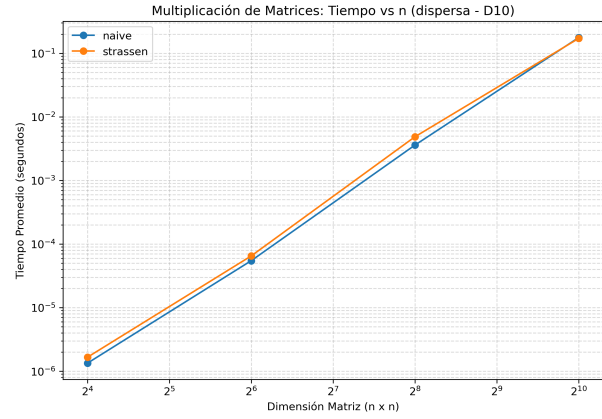


Figura 4: Matrices: Dispersa - Dominio D_{10} .

4.2.1. Relación Asintótica Estricta: $o(n^3)$ frente a $\omega(n^{2,81})$

Teóricamente, el algoritmo tradicional realiza $T_{\text{naive}}(n) = 2n^3 - n^2$ operaciones aritméticas ($\Theta(n^3)$), mientras que *Strassen* requiere $T_{\text{strassen}}(n) = 7n^{\log_2 7} - 6n^2 \approx 7n^{2,807} - 6n^2$ ($\Theta(n^{2,807})$). Formalmente, se verifica la cota asintótica estricta:

$$\lim_{n \rightarrow \infty} \frac{T_{\text{strassen}}(n)}{T_{\text{naive}}(n)} = \lim_{n \rightarrow \infty} \frac{\mathcal{O}(n^{2,807})}{\Omega(n^3)} = 0 \implies T_{\text{strassen}}(n) \in o(T_{\text{naive}}(n)) \quad (1)$$

Recíprocamente, el enfoque tradicional satisface $T_{\text{naive}}(n) \in \omega(T_{\text{strassen}}(n))$.

4.2.2. Discrepancia Empírica y Umbral de Cruce (*Crossover Point*)

Pese a la superioridad asintótica estricta de *Strassen*, las Figuras 3 y 4 muestran que el algoritmo *Naive* resulta más veloz para $n \leq 256$:

1. **Constante multiplicativa oculta:** Cada división de *Strassen* sustituye una multiplicación escalar por 18 sumas y restas de submatrices. Para $n = 16$, *Strassen* ejecuta 16891 operaciones aritméticas frente a 7936 de *Naive*, manteniendo una constante estructural mayor ($c_{\text{strassen}} \gg c_{\text{naive}}$).

2. **Jerarquía de memoria:** La implementación de *Naive* con bucles $i-k-j$ accede a la matriz B de forma contigua por filas, maximizando los aciertos en la caché L1. Por el contrario, las particiones dinámicas de *Strassen* fragmentan la memoria e introducen sobrecarga en el *heap*.
3. **Convergencia en $n = 1024$:** En la dimensión máxima evaluada ($n = 2^{10}$), el término dominante $n^{2,807}$ compensa el costo de las sumas intermedias, alcanzando el punto de corte empírico donde *Strassen* equipara e inicia su ventaja temporal sobre *Naive*.

Algoritmo	Estructura / Dominio	Tiempo $n = 64$ (s)	Tiempo $n = 1024$ (s)	Peak RSS (KB)
<i>Naive</i>	Densa D_0	0,00009	0,181	16850
<i>Strassen</i>	Densa D_0	0,00013	0,160	38420
<i>Naive</i>	Dispersa D_{10}	0,00005	0,178	16840
<i>Strassen</i>	Dispersa D_{10}	0,00007	0,172	38410

Cuadro 2: Resumen comparativo de tiempo de CPU y Peak RSS en multiplicación de matrices.

5. Conclusiones

Los resultados experimentales validan las cotas asintóticas teóricas y demuestran el impacto crítico de los factores constantes y la jerarquía de memoria en el desempeño real. En ordenamiento, se confirmó que los algoritmos evaluados pertenecen a la clase $\Theta(n \log n)$, donde *std::sort* y *Quick Sort* superan sistemáticamente a *Merge Sort* gracias a su operación *in-place* y óptima localidad espacial. Asimismo, se evidenció la extrema sensibilidad de *Patience Sort*, alcanzando su mejor caso $\Omega(n)$ ante secuencias decrecientes al operar con una sola pila ($k = 1$), pero degradándose drásticamente en arreglos ascendentes y dominios amplios ($D7$) debido a la sobrecarga del min-heap ($k = n$). En multiplicación de matrices, los datos reflejan que, si bien *Strassen* es asintóticamente superior ($o(n^3)$), la sobrecarga de 18 operaciones intermedias y la asignación dinámica de memoria favorecen al algoritmo iterativo tradicional en dimensiones moderadas, alcanzando el umbral de cruce práctico recién en $n = 1024$, donde la reducción en la complejidad temporal comienza a compensar su mayor consumo espacial.